

Transformada de Hough

Joaquin Koifman, Paz Lavenia, Ema Sapirstein e Ingrid von Foerster

8 de junio de 2026

Transformada de Hough

Es una técnica para la detección de figuras como líneas y círculos en imágenes digitales de manera rápida y eficiente partiendo de su mapa de bordes.

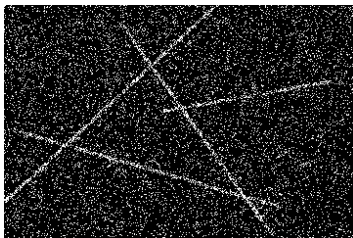
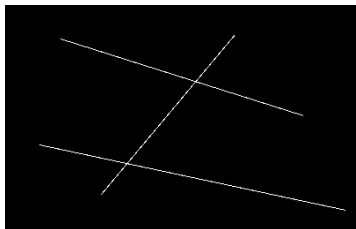


El desafío que busca resolver Hough

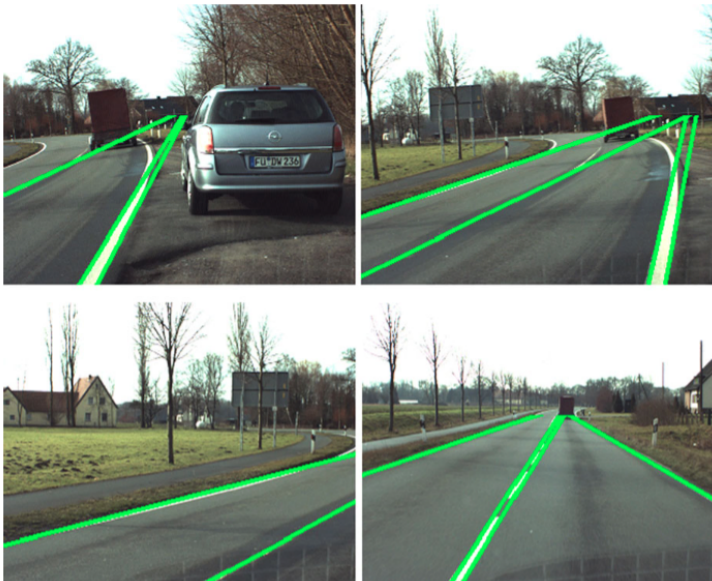
Para identificar formas como líneas rectas, el algoritmo debe ser robusto ante:

- **Ruido y datos irrelevantes:** Bordes secundarios que ensucian la escena.
- **Datos incompletos:** Líneas interrumpidas por otros objetos en la imagen.

La Transformada de Hough encuentra formas en una imagen convirtiendo cada píxel de borde en una curva matemática dentro del espacio de parámetros.



Localizar Lineas en una Imagen

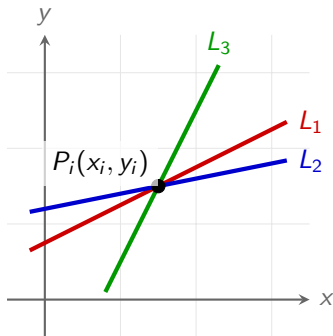


Espacio de la Imagen vs. Espacio de Parámetros

Partimos de la ecuación explícita de la recta: $y = mx + c$

Espacio de la imagen (x, y)

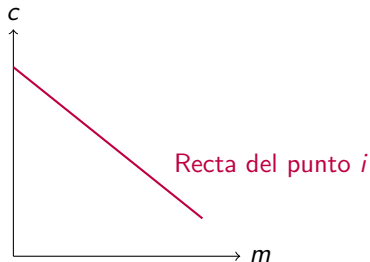
Por un punto (x_i, y_i) pasan infinitas rectas determinadas por m y c .



Espacio de parámetros (m, c)

Despejando: $c = -x_i \cdot m + y_i$.

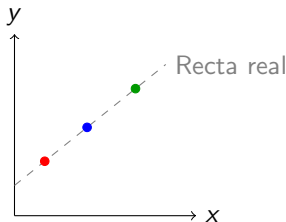
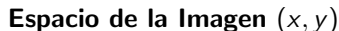
Las variables ahora son (m, c) .



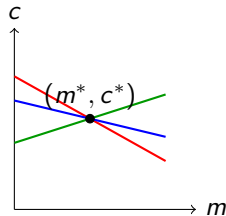
Las rectas en el espacio de la imagen son puntos en el espacio de parámetros y viceversa.

El Principio de Colinealidad

¿Cómo sabemos si varios puntos en la imagen forman la misma recta?



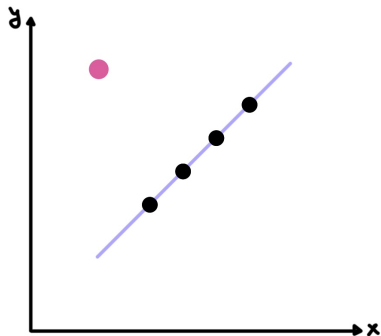
- Puntos alineados que comparten la misma pendiente e intersección.



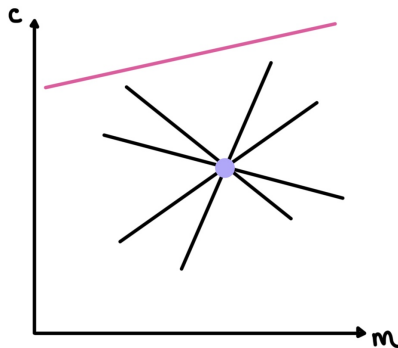
- Las rectas de cada punto se **intersecan** exactamente en la solución común (m^*, c^*) .

¿Cómo distinguimos el ruido?

Espacio de la Imagen (x, y)



Espacio de Parámetros (m, c)



Algoritmo de detección

Para procesar esto digitalmente, la computadora cuantiza el espacio de parámetros en una matriz discreta.

Algoritmo de detección:

- 1 Se cuantiza el espacio de parámetros (m, c) .
- 2 Se inicializa la matriz acumuladora en cero: $A(m, c) = 0 \forall m, c$.
- 3 Por cada píxel de borde (x_i, y_i) , se calcula su recta en el espacio de parámetros.
- 4 $A(m, c) = A(m, c) + 1$ si (m, c) está en $c = -mx_i + y_i$.
- 5 Se buscan los **máximos locales** de la matriz.

Matriz Acumuladora

0	1	0
1	2	1
0	1	0

La celda con valor **3** representa la recta buscada.

Elección del tamaño de las celdas

Observación

Elegir el tamaño de las celdas acumuladoras es parte del problema: si son muy grandes, no distingue líneas distintas y, si son muy chicas, el algoritmo se vuelve sensible al ruido.

La Limitación del Infinito y el Espacio (ρ, θ)

Problema del espacio (m, c)

$m \in (-\infty, +\infty)$, por lo que es necesaria una matriz acumuladora muy grande o infinita, utilizando demasiada memoria y cómputo

Solución: Parametrización Normal o Polar

$$x \cdot \cos(\theta) + y \cdot \sin(\theta) = \rho$$

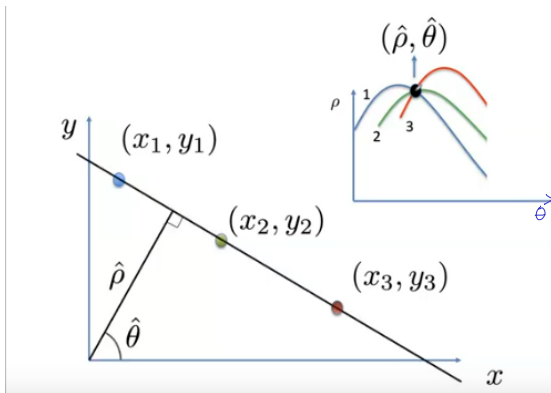
- ρ (Rho): Distancia más corta desde el origen hasta el punto. Acotada por el tamaño de la imagen.
- θ (Theta): Ángulo del vector ρ . $\theta \in [0, \pi]$ o $\theta \in [-90^\circ, 90^\circ]$.

*En el espacio (ρ, θ) , cada punto dibuja una **sinusoide**. Las intersecciones de estas curvas marcan las líneas rectas detectadas.*

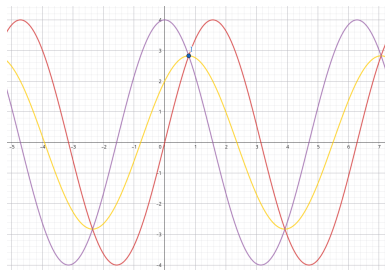
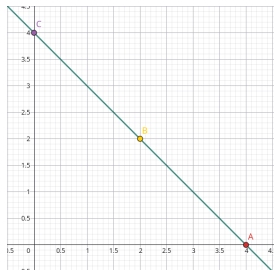
Espacio de la imagen vs espacio de coordenadas polares

¿Qué pasa ahora?

- Los puntos de la imagen son sinusoides en el espacio de parámetros.
- La intersección entre las sinusoides en el espacio de parámetros es una línea por la que pasan los puntos en la imagen.



Transformada de Hough de 3 puntos de una recta

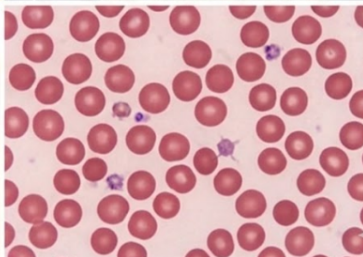


Ejemplo

recta : $y = -x + 4$

intersección de las sinusoides en: $\rho = \sqrt{8}$ y $\theta = 45^\circ$

Localizar Circulos en una Imagen

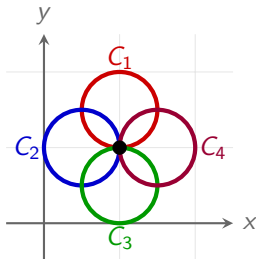


Transformada de Hough para círculos

Partimos de la ecuación: $(x - a)^2 + (y - b)^2 = r^2$ siendo r el radio del círculo y (a, b) su centro.

Espacio de la imagen (x, y)

Por un punto (x_i, y_i) pasan infinitos círculos determinados por a , b y r .

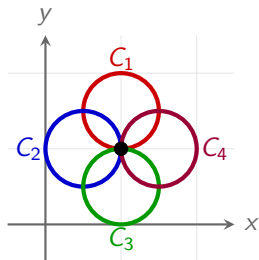


Caso r conocido

En este caso, el espacio de parámetros es de 2 dimensiones.

Espacio de la imagen (x, y)

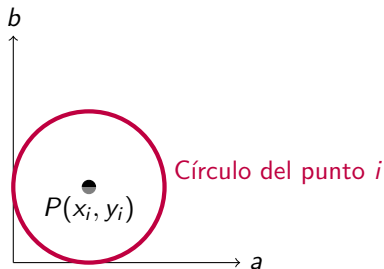
Por un punto (x_i, y_i) pasan infinitos círculos de radio r determinados por a , b .



Espacio de parámetros (a, b)

Despejando: $(a - x_i)^2 + (b - y_i)^2 = r^2$.

Las variables ahora son (a, b) .

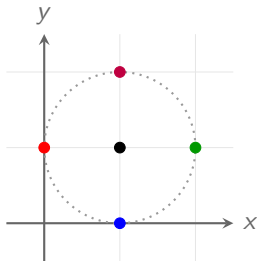


Los círculos en el espacio de la imagen son puntos en el espacio de parámetros y viceversa.

Puntos en un mismo círculo

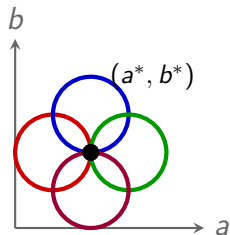
¿Cómo sabemos si varios puntos en la imagen forman el mismo círculo?

Espacio de la Imagen (x, y)



- Puntos que forman el mismo círculo.

Espacio de Parámetros (a, b)



- Los círculos de cada punto se **intersecan** exactamente en la solución común (a^*, b^*) .

Algoritmo de detección

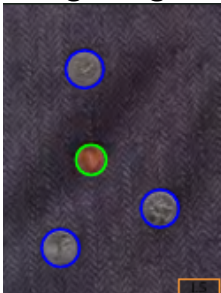
Para procesar esto digitalmente, la computadora cuantiza el espacio de parámetros en una matriz discreta.

Algoritmo de detección:

- 1 Se cuantiza el espacio de parámetros (a,b) .
- 2 Se inicializa la matriz acumuladora en cero: $A(a, b) = 0 \forall a, b$.
- 3 Por cada píxel de borde (x_i, y_i) , se calcula su círculo de radio r en el espacio de parámetros.
- 4 $A(a, b) = A(a, b) + 1$ si (a, b) está en $(x_i - a)^2 + (y_i - b)^2 = r^2$.
- 5 Se buscan los **máximos locales** de la matriz.

Ejemplo Monedas

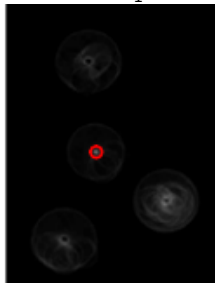
Imagen original



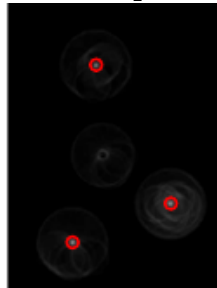
Bordes



$r = r_1$



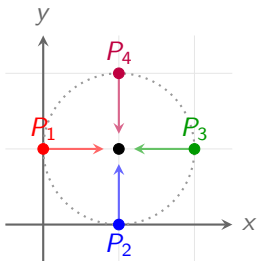
$r = r_2$



Detección de círculos optimizada con r conocido

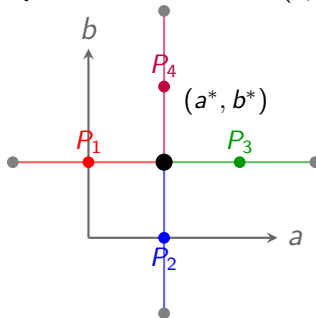
Si en el espacio de la imagen buscamos la dirección del borde podemos optimizar el algoritmo de detección. La dirección la obtenemos calculando el gradiente.

Espacio de la Imagen (x, y)



- Puntos que forman el mismo círculo.

Espacio de Parámetros (a, b)



- Los segmentos de cada punto se intersectan en (a^*, b^*) .

Algoritmo de detección

Para procesar esto digitalmente, la computadora cuantiza el espacio de parámetros en una matriz discreta. Se utiliza la dirección de cada punto del borde ϕ_i calculada utilizando el gradiente.

Algoritmo de detección:

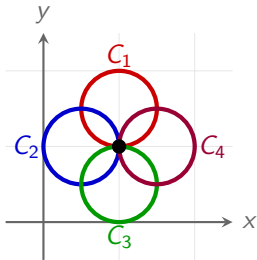
- 1 Se cuantiza el espacio de parámetros (a,b) .
- 2 Se inicializa la matriz acumuladora en cero: $A(a, b) = 0 \forall a, b$.
- 3 Por cada píxel de borde (x_i, y_i) , se calcula su círculo de radio r en el espacio de parámetros $A(a, b) = A(a, b) + 1$ si $a = x_i \pm r \cdot \cos(\phi_i)$ y $b = y_i \pm r \cdot \sin(\phi_i)$.
- 4 Se buscan los **máximos locales** de la matriz.

Caso r desconocido

En este caso el espacio de parámetros es de 3 dimensiones.

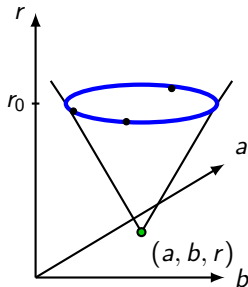
Espacio de la imagen (x, y)

Por un punto (x_i, y_i) pasan infinitos círculos determinados por a , b y r .



Espacio de parámetros (a, b, r)

Despejando: $(a - x_i)^2 + (b - y_i)^2 = r^2$.
Las variables ahora son (a, b, r) .



Los puntos en el espacio de la imagen son conos en el espacio de parámetros y conos en el espacio de parámetros son círculos en el espacio de la imagen.

Algoritmo de detección

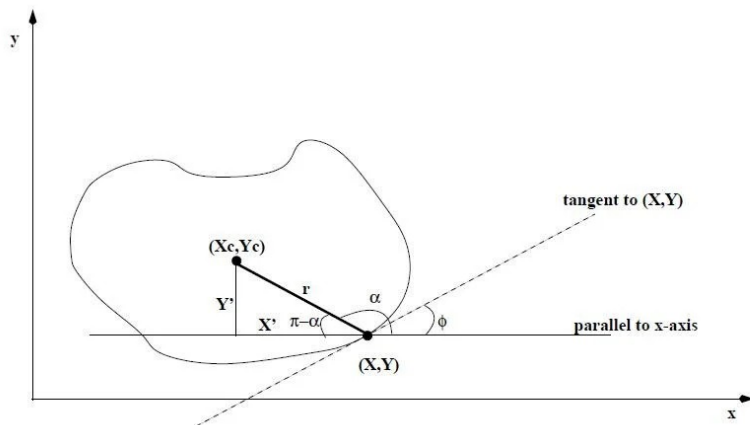
Para procesar esto digitalmente, la computadora cuantiza el espacio de parámetros en una matriz tridimensional.

Algoritmo de detección:

- ① Se cuantiza el espacio de parámetros (a, b, r) .
- ② Se inicializa la matriz tridimensional acumuladora en cero:
 $A(a, b, r) = 0 \forall a, b, r$.
- ③ Por cada píxel de borde (x_i, y_i) , se calcula su cono en el espacio de parámetros.
- ④ $A(a, b, r) = A(a, b, r) + 1$ si (a, b, r) está en $(x_i - a)^2 + (y_i - b)^2 = r^2$.
- ⑤ Se buscan los **máximos locales** de la matriz.
 - Para optimizar el algoritmo en lugar de analizar todos los r , se define un rango $[r_{min}, r_{max}]$

Transformada de Hough Generalizada

¿Qué pasa cuando el objeto que quiero detectar no es ni una recta ni un círculo?



Transformada de Hough Generalizada

Para procesar formas arbitrarias, se utiliza un modelo basado en una tabla ϕ y una matriz acumuladora.

Algoritmo de detección:

Se define un punto de referencia (x_c, y_c) asociado a la forma que se desea detectar.

Se inicializa la matriz acumuladora:

$$A(x_c, y_c) = 0 \quad \forall (x_c, y_c)$$

Para cada punto de borde (x_i, y_i, ϕ_i) :

- Se consulta la tabla ϕ
- Para cada entrada (r_k, α_k) , calcular:

$$x_c = x_i + r_k \cos(\alpha_k)$$

$$y_c = y_i + r_k \sin(\alpha_k)$$

Incrementar el acumulador:

$$A(x_c, y_c) = A(x_c, y_c) + 1$$

Bibliografía oficial de la materia

Gonzales,Woods-Digital Image Processing 4th Edition

Visualizadores:

<https://dersmon.github.io/HoughTransformationDemo/>

<https://liquiddandruff.github.io/hough-transform-visualizer/>

Videos explicativos:

Lineas y círculos

Formas generales